

การดำเนินการ Async

createAsyncThunk และการรวม API

หลักสูตร React-Redux · โปรเจกต์ AcadeMate

Dr.Watcharin Sarachai

ทำไมต้องใช้ Async?

Redux เป็น sync โดยค่าเริ่มต้น — เรียนรู้ว่าทำไม
จึงต้องใช้ middleware

createAsyncThunk

ยูทิลิตี้ RTK สำหรับห่อ async API calls เป็น
thunks

extraReducers

จัดการสถานะวงจรชีวิต pending / fulfilled /
rejected

Labภาคปฏิบัติ

เชื่อมต่อ AcadeMate กับ mockapi.io REST
API

Session 4 — เป้าหมายการเรียนรู้

เมื่อสิ้นสุด Session นี้ผู้เรียนจะสามารถ:

ส่วนที่ 1 — ทำไมต้องใช้ Async?

เข้าใจว่าทำไม Redux จึงต้องการ middleware และ Thunk ทำงานภายในอย่างไร

ส่วนที่ 2 — createAsyncThunk

กำหนด async thunks, เชื่อมต่อ REST API, จัดการสถานะวงจรชีวิต

ส่วนที่ 3 — extraReducers

เขียน reducers สำหรับ pending/fulfilled/rejected และอัปเดตสถานะ UI

Lab 4 — AcadeMate API

แทนที่ข้อมูล hard-code ด้วย CRUD calls จริงไปยัง mockapi.io

ส่วนที่ 4 — การจัดการข้อผิดพลาด

แสดง loading spinner, แบนเนอร์ข้อผิดพลาด และ retry logic

ทำไมต้องใช้ Async?

Redux เป็น synchronous โดยค่าเริ่มต้น — การ dispatch action จะอัปเดต store ทันที
การเรียก API, timer และการอ่านไฟล์เป็น asynchronous จึงต้องมีตัวเชื่อม

Redux เป็น Synchronous — ปัญหาที่เกิดขึ้น

ทุกการเรียก dispatch เกิดขึ้นทันที แต่การดึงข้อมูลจากเซิร์ฟเวอร์ต้องใช้เวลา หากไม่มี middleware ไม่สามารถทำได้

❌ วิธีนี้ไม่ถูกต้อง

```
// This does NOT work in Redux
function fetchStudents() {
  return async (dispatch) => {
    // Can't dispatch a function
    // - Redux only accepts plain
    // objects as actions!
  };
}

dispatch(fetchStudents());
// ❌ Error: Actions must be plain
// objects. Use middleware.
```

✅ ด้วย Redux Thunk Middleware

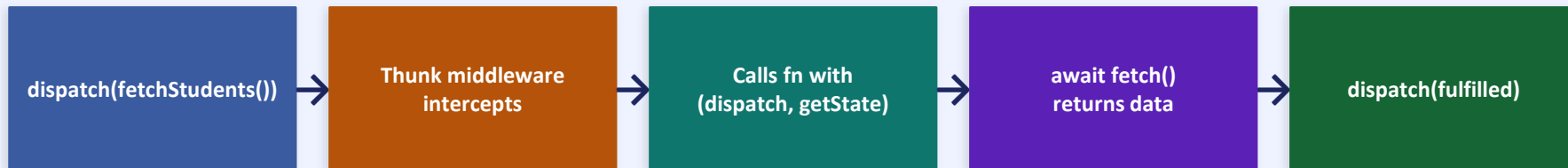
```
// Redux Toolkit includes thunk
// middleware automatically.
// RTK's configureStore() sets
// it up with no extra config.

import { createAsyncThunk } from '@reduxjs/toolkit';

export const fetchStudents =
  createAsyncThunk(
    'students/fetchAll',
    async () => {
      const res = await fetch(URL);
      return res.json();
    }
  );
```

การทำงานของ Redux Thunk Middleware

Thunk จะ intercept dispatch ถ้า dispatch เป็น function, และ Thunk จะเรียกมันด้วย (dispatch, getState) แต่ถ้า dispatch เป็น plain object ก็จะถูกส่งต่อไปตรงๆ



```
// Thunk middleware pseudocode (simplified)
const thunkMiddleware = store => next => action => {
  if (typeof action === 'function') {
    // It's a thunk - call it with dispatch and getState
    return action(store.dispatch, store.getState);
  }
  // It's a plain object - pass through normally
  return next(action);
};

// RTK's configureStore() adds this automatically:
const store = configureStore({ reducer: rootReducer });
//                               ↑ thunk middleware included by default
```

อธิบาย: การทำงานของ Redux Thunk Middleware



ฟังก์ชันซ้อน 3 ชั้น — รับ store → รับ next → รับ action

เหมือนด่านตรวจ 3 ด่าน: ด่านแรกรู้จัก store ด่านสองรู้จัก middleware ถัดไป ด่านสุดท้ายรับ action ที่ส่งมา



ถ้า action เป็น "ฟังก์ชัน" → ให้มันทำงานก่อน

เหมือนยามที่เห็นของจดหมายพิเศษ (thunk) → เปิดออกอ่านและดำเนินการ เช่น เรียก API แล้วค่อย dispatch ผลลัพธ์



ถ้า action เป็น object ธรรมดา → ส่งต่อเลย ไม่แทรกแซง

เหมือนสายพานลำเลียง — ถ้าไม่ใช่ thunk ก็ปล่อยผ่านไปหา reducer ได้ตามปกติ ไม่มีขั้นตอนพิเศษ

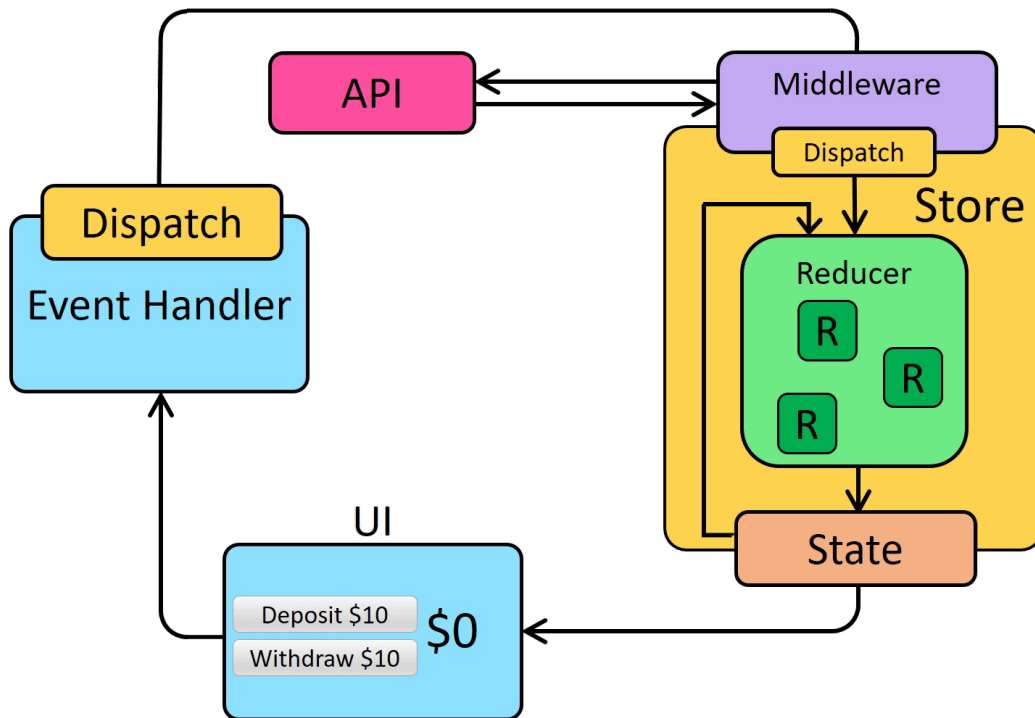


configureStore() ของ RTK ติดตั้ง thunk ให้อัตโนมัติ

ไม่ต้องเพิ่มโค้ดใด ๆ เอง — แค่ใช้ configureStore() ก็พร้อมใช้ thunk ได้ทันที เหมือน Wi-Fi ที่เปิดมาพร้อมใช้อยู่แล้ว

Middleware และกระบวนการข้อมูล Redux

Middleware เพิ่มขั้นตอนพิเศษตอนต้นของ *dispatch*. That way, middleware can run logic like HTTP requests, then dispatch actions. That makes the async data flow look like this:



createAsyncThunk — สาม Actions ที่สร้างอัตโนมัติ

ทุกครั้งที่เรียก createAsyncThunk — Redux จะสร้าง action type ให้อัตโนมัติ 3 ตัว ตามสถานะของ API call

 **pending** `students/fetchAll/pending`

 **เกิดขึ้นเมื่อ:** ทันทีที่กด fetch — ก่อน API จะเริ่มทำงาน

 **ใช้เพื่อ:** แสดง loading spinner และล็อกปุ่มไม่ให้กดซ้ำ

 **fulfilled** `students/fetchAll/fulfilled`

 **เกิดขึ้นเมื่อ:** API ตอบกลับสำเร็จ — ได้ข้อมูลกลับมาแล้ว

 **ใช้เพื่อ:** นำข้อมูลไปเก็บใน state และซ่อน spinner

 **rejected** `students/fetchAll/rejected`

 **เกิดขึ้นเมื่อ:** API ล้มเหลว — network error หรือ server error

 **ใช้เพื่อ:** ซ่อน spinner และแสดงข้อความแจ้งข้อผิดพลาด

createAsyncThunk

กำหนด async thunks, เชื่อมต่อกับ slice และ dispatch จาก components

อ่านโค้ด createAsyncThunk แบบทีละส่วน

โค้ดหน้าถัดไปมี 2 argument หลัก — เข้าใจแต่ละส่วนก่อน แล้วอ่านโค้ดจะง่ายขึ้นทันที

```
createAsyncThunk(  
  'students/fetchAll',  
  async (_, thunkAPI) => {  
    try {  
      const res = await fetch(url);  
      if (!res.ok)  
        throw new Error('Server error');  
      const data = await res.json();  
      return data;  
    } catch (err) {  
      return thunkAPI  
        .rejectWithValue(  
          err.message);  
    }  
  }  
);
```

① Argument 1 — ชื่อ action prefix

'students/fetchAll' → Redux จะสร้าง action type อัตโนมัติ 3 ตัว:
pending / fulfilled / rejected

② Argument 2 — async function

รับ (arg, thunkAPI) ทำงาน async ภายใน — เรียก API แล้ว return ผลลัพธ์

③ return data — สำเร็จ

ค่าที่ return จะกลายเป็น action.payload ใน .fulfilled → นำไปเก็บใน state ได้เลย

④ rejectWithValue() — ล้มเหลว

ส่ง error กลับเป็น action.payload ใน .rejected → แสดง error ใน UI ได้

thunkAPI คืออะไร — เครื่องมือที่ซ่อนอยู่ใน async function

object ที่ Redux ส่งให้เป็น argument ที่ 2 ของ async function — มี 6 เครื่องมือให้ใช้ได้เลย

ตำแหน่งในโค้ด: `createAsyncThunk('prefix', async (arg, thunkAPI) => { ... })`

thunkAPI.dispatch → (action) => void
ส่ง action อื่นต่อเนื่องได้ภายใน thunk เช่น dispatch หลังจาก fetch สำเร็จ

thunkAPI.getState → () => RootState
อ่านค่า state ปัจจุบันจาก Redux store ได้ทันที ใช้ตรวจสอบข้อมูลก่อน fetch

thunkAPI.rejectWithValue → (value) => payload
ส่ง error กลับเป็น action.payload ใน .rejected — ควรใช้แทน throw เสมอ

thunkAPI.fulfillWithValue → (value) => payload
เหมือน return แต่แนบ meta เพิ่มได้ — ใช้เมื่อต้องการข้อมูลพิเศษใน .fulfilled

thunkAPI.signal → AbortSignal
ใช้ยกเลิก fetch กลางคันได้ (เช่น ผู้ใช้ออกจากหน้าก่อน API ตอบ)

thunkAPI.requestId → string
ID unique ของ request นี้ — ใช้เปรียบเทียบเพื่อกันผลลัพธ์เก่าเขียนทับใหม่

createAsyncThunk — ไวยากรณ์ครบล้วน

createAsyncThunk รับอาร์กิวเมนต์สองแบบ: string ชนิด action และ payload creator function (async)

```
import { createAsyncThunk } from '@reduxjs/toolkit';

// Argument 1: action type prefix → 'students/fetchAll'
// Argument 2: async payload creator – receives (arg, thunkAPI)
export const fetchStudents = createAsyncThunk(
  'students/fetchAll', // prefix
  async (_, thunkAPI) => { // _ = no argument passed
    try {
      const response = await fetch(
        'https://YOUR_ID.mockapi.io/api/v1/students'
      );
      if (!response.ok) throw new Error('Server error');
      const data = await response.json();
      return data; // becomes action.payload on fulfilled
    } catch (err) {
      return thunkAPI.rejectWithValue(err.message); // becomes action.payload on rejected
    }
  }
);

// Auto-generates three action creators:
// fetchStudents.pending → 'students/fetchAll/pending'
// fetchStudents.fulfilled → 'students/fetchAll/fulfilled'
// fetchStudents.rejected → 'students/fetchAll/rejected'
```

ตั้งค่า mockapi.io สำหรับ AcadeMate

mockapi.io ให้บริการ REST API แบบ hosted ฟรี — ไม่ต้องมี backend เราจะสร้างสาม resources ที่สะท้อน Redux slices

1 ไปที่ mockapi.io → สร้างโปรเจกต์ใหม่ → ตั้งชื่อว่า 'academate'

2 เพิ่ม resource: students (fields: name, studentId, major, gpa)

3 เพิ่ม resource: courses (fields: title, code, credits, instructor)

4 เราไม่เพิ่ม grades (fields: studentId, courseId, score, letterGrade) เว็บบ mockapi.io แบบฟรีให้สร้างได้สูงสุด 2 resources เท่านั้น (ใช้แบบเดิม local store)

5 คัดลอก base URL ของคุณ: `https://{YOUR_ID}.mockapi.io/api/v1`

6 เพิ่มข้อมูลเริ่มต้น 5-10 รายการในแต่ละ resource ผ่าน UI ของ mockapi.io

รูปแบบ base URL: `https://{YOUR_ID}.mockapi.io/api/v1/{resource}` → เช่น `/students` `/courses` `/grades`

studentsSlice — Async Thunks (1/2: fetch และ add)

สร้าง async thunks ในไฟล์ thunks.js แยกต่างหากควบคู่กับ slice เพื่อการจัดระเบียบโค้ดที่ดี

```
// src/features/students/studentsThunks.js
import { createAsyncThunk } from '@reduxjs/toolkit';
const BASE = 'https://69da9b2226585bd92dd400ca.mockapi.io/api/v1';

export const fetchStudents = createAsyncThunk(
  'students/fetchAll',
  async (_, { rejectWithValue }) => {
    const res = await fetch(`${BASE}/students`);
    if (!res.ok) return rejectWithValue('Failed to fetch');
    return res.json();
  }
);

export const addStudentAsync = createAsyncThunk(
  'students/addOne',
  async (studentData, { rejectWithValue }) => {
    const res = await fetch(`${BASE}/students`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(studentData),
    });
    if (!res.ok) return rejectWithValue('Failed to add');
    return res.json(); // server returns the created record with id
  }
);
```

studentsSlice — Async Thunks (2/2: update และ delete)

```
// src/features/students/studentsThunks.js (continued)
export const updateStudentAsync = createAsyncThunk(
  'students/updateOne',
  async ({ id, ...changes }, { rejectWithValue }) => {
    const res = await fetch(`${BASE}/students/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(changes),
    });
    if (!res.ok) return rejectWithValue('Failed to update');
    return res.json();
  }
);

export const deleteStudentAsync = createAsyncThunk(
  'students/deleteOne',
  async (id, { rejectWithValue }) => {
    const res = await fetch(`${BASE}/students/${id}`, {
      method: 'DELETE',
    });
    if (!res.ok) return rejectWithValue('Failed to delete');
    return id; // return the id so the reducer can remove it
  }
);
```

extraReducers

จัดการสถานะวงจรชีวิต pending / fulfilled / rejected ภายใน slice

extraReducers คืออะไร และทำไมต้องใช้?

ก่อนเรียนโค้ด — เข้าใจความแตกต่างระหว่าง reducers กับ extraReducers

reducers — action ภายใน slice เดียวกัน

-  สร้าง action creator ได้เลยภายใน slice
-  ใช้กับ action แบบ synchronous ทันที

```
dispatch(setFilter('active'))
```

extraReducers — รับฟัง action จากภายนอก

-  รับฟัง action จาก thunk หรือ slice อื่น
-  ใช้กับ async — ผูก pending/fulfilled/rejected

```
builder.addCase(fetchStudents.fulfilled, ...)
```

ลำดับที่เกิดขึ้นเมื่อ `dispatch(fetchStudents())`

① `dispatch(fetchStudents())`
เรียก thunk function



② Redux สร้าง action type
pending / fulfilled / rejected



③ extraReducers รับ action
อัปเดต state ตามผลลัพธ์



สรุป:

reducers = จัดการ action ของตัวเอง (sync) | **extraReducers** = รับฟัง thunk actions (async) แล้วอัปเดต state

extraReducers จัดการ actions ที่กำหนดนอก slice — รวมถึง action ที่สร้างอัตโนมัติสามแบบจาก thunk

```
// src/features/students/studentsSlice.js
import { createSlice } from "@reduxjs/toolkit";
import { fetchStudents, addStudentAsync } from "./studentsThunks";

const studentsSlice = createSlice({
  name: "students",
  initialState: {
    list: [],
    status: "idle", // 'idle' | 'loading' | 'succeeded' | 'failed'
    error: null,
  },
  reducers: {},
  extraReducers: (builder) => {
    builder
      .addCase(fetchStudents.pending, (state) => { state.status = "loading"; state.error = null; })
      .addCase(fetchStudents.fulfilled, (state, action) => { state.status = "succeeded"; state.list = action.payload; })
      .addCase(fetchStudents.rejected, (state, action) => { state.status = "failed"; state.error = action.payload; });
  }
});
```

extraReducers — เพิ่ม, อัปเดต, ลบ Cases (2/3)

เชื่อม `.addCase()` เพิ่มเติมสำหรับ `addStudentAsync`, `updateStudentAsync` และ `deleteStudentAsync`

```
// Append these cases to the builder chain above
.addCase(addStudentAsync.fulfilled, (state, action) => {
  state.list.push(action.payload); // server returns full record with id
})
.addCase(updateStudentAsync.fulfilled, (state, action) => {
  const idx = state.list.findIndex((s) => s.id === action.payload.id);
  if (idx !== -1) state.list[idx] = action.payload;
})
.addCase(deleteStudentAsync.fulfilled, (state, action) => {
  state.list = state.list.filter((s) => s.id !== action.payload);
})
// Generic loading/error for all mutating thunks
.addMatcher(
  (action) => action.type.endsWith("/pending") && action.type.startsWith("students/"),
  (state) => {
    state.status = "loading";
  },
)
```

extraReducers — เพิ่ม, อัปเดต, ลบ Cases (3/3)

เชื่อม `.addCase()` เพิ่มเติมสำหรับ `addStudentAsync`, `updateStudentAsync` และ `deleteStudentAsync`

```
// Append these cases to the builder chain above
.addMatcher(
  (action) => action.type.endsWith("/rejected") && action.type.startsWith("students/"),
  (state, action) => {
    state.status = "failed";
    state.error = action.payload;
  },
);

});

});

export default studentsSlice.reducer;
```

ใช้จับ action หลาย type ด้วยเงื่อนไขเดียว แทนการเขียน .addCase() ซ้ำหลายรอบ

■ .addMatcher() คืออะไร?

เป็น method ของ **ActionReducerMapBuilder** ใน Redux Toolkit

รับ **matcher** (ฟังก์ชันเงื่อนไข) และ **reducer** (ฟังก์ชัน handler)

■ เมื่อไหร่ควรใช้?

- ✓ จัดการ action หลาย type ที่มี logic เดียวกัน (เช่น ทุก rejected)
- ✓ ใช้ pattern matching แทน switch-case ยาว ๆ
- ✓ ใช้ร่วมกับ helper เช่น **isAnyOf**, **isPending**, **isFulfilled**, **isRejected**

ใช้ isRejected จาก RTK เพื่อ match ทุก rejected action ใน slice เดียว

```
// นำเข้า isRejected จาก @reduxjs/toolkit
import { isRejected } from "@reduxjs/toolkit";

const studentsSlice = createSlice({
  extraReducers: (builder) => {
    builder
      .addCase(fetchStudents.fulfilled, (state, action) => { ... })
      // ใช้ addMatcher แทน addCase หลายๆ ตัวสำหรับ rejected
      .addMatcher(isRejected(fetchStudents, addStudentAsync, deleteStudentAsync),
        (state, action) => {
          state.status = "failed";
          state.error = action.payload;
        })
  }
});
```

ฟิลต์ status — รูปแบบสี่สถานะ

ฟิลต์ status ขับเคลื่อน async UI ทั้งหมด ให้ใช้ค่า string สี่แบบเหล่านี้เสมอ

'idle'

สถานะเริ่มต้น — ยังไม่มีการส่งคำขอ

ตอน render แรก ก่อนผู้ใช้เรียกดึงข้อมูล

'loading'

มีคำขออยู่ระหว่างดำเนินการ — แสดง spinner, ปิดใช้งานปุ่ม

ตั้งค่าใน .pending handler

'succeeded'

ข้อมูลมาแล้ว — render ลิสต์/ตาราง

ตั้งค่าใน .fulfilled handler

'failed'

คำขอล้มเหลว — แสดงแบนเนอร์ข้อผิดพลาด เปิดใช้งาน retry

ตั้งค่าใน .rejected handler

Lab 4 — เชื่อมต่อ AcadeMate กับ API

แทนที่ Redux state แบบ hard-code ด้วย CRUD calls จริงไปยัง mockapi.io endpoint

Lab 4 — ภาพรวมและไฟล์ที่เปลี่ยนแปลง

เราจะแก้ไข 4 ไฟล์และสร้าง 1 ไฟล์ใหม่ UI components ทั้งหมดคงเดิม — เฉพาะ data layer ที่เปลี่ยน

สร้าง

`src/features/students/studentsThunks.js`

async thunks ทั้ง 4 แบบ (fetch, add, update, delete)

อัปเดต

`src/features/students/studentsSlice.js`

เพิ่มฟิลด์ status/error + extraReducers builder

อัปเดต

`src/components/StudentTable.jsx`

Dispatch async thunks แทน sync actions

อัปเดต

`src/components/AddStudentForm.jsx`

Dispatch addStudentAsync แทน addStudent

อัปเดต

`src/App.jsx`

Dispatch fetchStudents เมื่อ component mount

Lab 4 — ขั้นตอนที่ 1: อัปเดต studentsSlice.js

ขั้นตอนที่ 1: เพื่อ field ใน initial state ด้วยฟิลด์ status และ error แล้วเชื่อม extraReducers

```
// src/features/students/studentsSlice.js — Session 4 update
import { createSlice } from '@reduxjs/toolkit';
import {
  fetchStudents, addStudentAsync,
  updateStudentAsync, deleteStudentAsync,
} from './studentsThunks';
const studentsSlice = createSlice({
  name: 'students',
  initialState: {
    list: [],
    status: 'idle',    // NEW: tracks async state
    error: null,       // NEW: holds error message if failed
  },
  reducers: {},
  extraReducers: builder => {
    builder
      .addCase(fetchStudents.pending, state => { state.status = 'loading'; state.error = null; })
      .addCase(fetchStudents.fulfilled, (state, { payload }) => { state.status = 'succeeded'; state.list = payload; })
      .addCase(fetchStudents.rejected, (state, { payload }) => { state.status = 'failed'; state.error = payload; })
      .addCase(addStudentAsync.fulfilled, (state, { payload }) => { state.list.push(payload); })
      .addCase(updateStudentAsync.fulfilled, (state, { payload }) => {
        const i = state.list.findIndex(s => s.id === payload.id);
        if (i !== -1) state.list[i] = payload;
      })
      .addCase(deleteStudentAsync.fulfilled, (state, { payload }) => {
        state.list = state.list.filter(s => s.id !== payload.id);
      });
  },
});
export default studentsSlice.reducer;
```

Lab 4 — ขั้นตอนที่ 2: ดึงข้อมูลเมื่อ Mount ใน App.jsx

ขั้นตอนที่ 2: App.jsx เรียกโหลดข้อมูลเริ่มต้นด้วย useEffect + fetchStudents

```
// src/App.jsx - Session 4: fetch on mount
import { useEffect } from 'react';
import { useDispatch } from 'react-redux';
import { fetchStudents } from '../features/students/studentsThunks';
import GpaSummary from '../components/GpaSummary';
import AddStudentForm from '../components/AddStudentForm';
import StudentTable from '../components/StudentTable';
import './App.css';

function App() {
  const dispatch = useDispatch();

  useEffect(() => {
    dispatch(fetchStudents()); // load data from API when app starts
  }, [dispatch]);           // runs once on mount

  return (
    <div className="app-container">
      <header className="app-header"><h1>AcadeMate</h1></header>
      <main className="app-main">
        <GpaSummary />
        <AddStudentForm />
        <StudentTable />
      </main>
    </div>
  );
}
export default App;
```

Lab 4 — ขั้นตอนที่ 3: AddStudentForm.jsx — Async Dispatch

ขั้นตอนที่ 3: แทนที่ `dispatch(addStudent(...))` ด้วย `dispatch(addStudentAsync(...))`

Session 3 (sync)

```
import { addStudent }
  from '../features/students/studentsSlice';

const onSubmit = e => {
  e.preventDefault();
  dispatch(addStudent({
    id: Date.now(),
    ...form,
    gpa: parseFloat(form.gpa)
  }));
  setForm(EMPTY);
};
```

Session 4 (async)

```
import { addStudentAsync }
  from '../features/students/studentsThunks';

const onSubmit = async e => {
  e.preventDefault();
  await dispatch(addStudentAsync({
    ...form,
    gpa: parseFloat(form.gpa)
    // No id: server assigns it
  }));
  setForm(EMPTY);
};
```

ความแตกต่างหลัก: ไม่สร้าง `id: Date.now()` บน client อีกต่อไป — เซิร์ฟเวอร์กำหนด ID จริงที่คงทน นอกจากนี้ยัง `await dispatch` เพื่อตอบสนองต่อความสำเร็จหรือความล้มเหลว

Lab 4 — ขั้นตอนที่ 4: StudentTable.jsx — Async Delete และ Edit

ขั้นตอนที่ 4: แทนที่ deleteStudent/updateStudent แบบ sync ด้วย async thunks JSX คงเดิม

```
// src/components/StudentTable.jsx — Session 4 changes (top of file)
import { useSelector, useDispatch } from 'react-redux';
import { selectAllStudents } from '../features/students/selectors';
import {
  deleteStudentAsync,
  updateStudentAsync,
} from '../features/students/studentsThunks'; // ← changed import

function StudentTable() {
  const students = useSelector(selectAllStudents);
  const dispatch = useDispatch();
  const [editing, setEditing] = useState(null);

  const handleDelete = (id) => {
    dispatch(deleteStudentAsync(id)); // ← was: dispatch(deleteStudent(id))
  };

  const handleEditSave = (updated) => {
    dispatch(updateStudentAsync(updated)); // ← was: dispatch(updateStudent(updated))
    setEditing(null);
  };

  // JSX return remains exactly the same as Session 3
```

การจัดการข้อผิดพลาดและ Loading UI

แสดง loading indicator, ข้อความข้อผิดพลาด และปุ่ม retry ตามฟิลด์ status

การอ่าน status ใน Components

Selectors สำหรับ status และ error ให้ components แยกสาขาตามสถานะ loading โดยไม่ต้อง thunk logic

เพิ่มใน selectors.js

```
// src/features/students/selectors.js
// (add these two selectors)

export const selectStudentsStatus =
  state => state.students.status;

export const selectStudentsError =
  state => state.students.error;
```

StudentTable.jsx — การ render แบบมีเงื่อนไข

```
const status = useSelector(selectStudentsStatus);
const error = useSelector(selectStudentsError);

if (status === 'loading')
  return <div className="spinner">Loading...</div>;

if (status === "failed") {
  return (
    <div className="error-banner">
      <p>Error: {error}</p>
      <button onClick={() => dispatch(fetchStudents())}>
        Retry
      </button>
    </div>
  );
}
```

การอ่าน status — การ Render ตาราง (succeeded)

Render ตารางเฉพาะเมื่อ status === 'succeeded' — ป้องกันการ map ผ่าน state ที่ว่างหรือกำลังโหลด

```
// StudentTable.jsx — success branch
const allStudents = useSelector(selectAllStudents);
const status = useSelector(selectStudentsStatus);
// Guard: don't render until data is ready
if (status !== "succeeded") return null;

return (
  <table className="student-table">
    <thead>
      <tr><th>Name</th><th>Major</th><th>GPA</th><th>Actions</th></tr>
    </thead>
    <tbody>
      {students.map(student => (
        <tr key={student.id}>
          <td>{student.name}</td>
          <td>{student.major}</td>
          <td>{student.gpa}</td>
          <td>
            <button onClick={() => setEditing(student)}>Edit</button>
            <button onClick={() => handleDelete(student.id)}>Delete</button>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
);
```


unwrap() คืออะไร และทำไมต้องใช้?

ปัญหาของ dispatch() ที่ resolve เสมอ และวิธีแก้ด้วย .unwrap()

■ ปัญหา — dispatch() resolve เสมอ แม้ Thunk จะ fail

createAsyncThunk dispatch **fulfilled** หรือ **rejected** แต่ Promise จาก dispatch() จะ resolve ทั้งสองกรณี ทำให้ **try/catch** ไม่สามารถจับ error ได้ ต้องเช็ค **.rejected.match()** เองทุกครั้ง

■ ทางออก — ใช้ .unwrap()

.unwrap() เป็น method พิเศษที่ RTK ต่อท้าย Promise จาก dispatch()

✓ fulfilled → คืนค่า **action.payload** โดยตรง ใช้ต่อได้เลย

✗ rejected → throw error จาก **rejectWithValue()** ให้ catch จับได้ทันที

■ ควรใช้เมื่อ

- ✓ ต้องการ **try/catch** ที่อ่านง่ายใน component แทนการเช็ค action type
- ✓ ต้องรู้ผลทันทีหลัง dispatch เช่น redirect หลัง save หรือ แสดง toast
- ✓ ต้องการแสดง error เฉพาะ component โดยไม่ต้องเก็บใน Redux state

ขั้นสูง: unwrap() — การจัดการผลลัพธ์ Thunk

dispatch() resolve เสมอ (ไม่เคย reject) — แม้จะล้มเหลว ใช้ .unwrap() เพื่อ rethrow เมื่อ reject และสามารถใช้ try/catch ใน component

```
// Without unwrap – dispatch always resolves
const result = await dispatch(addStudentAsync(formData));
if (addStudentAsync.rejected.match(result)) {
  setError('Failed to save'); // manual check required
}

// -----
// With .unwrap() – cleaner try/catch pattern
const onSubmit = async e => {
  e.preventDefault();
  try {
    await dispatch(addStudentAsync(formData)).unwrap();
    setForm(EMPTY);
    setSuccessMsg('Student saved!');
  } catch (err) {
    setError(err); // err === action.payload from rejectWithValue()
  }
};
```

ตรวจสอบความรู้

คำถาม:

นักพัฒนาเขียน extraReducers case นี้:

```
.addCase(fetchStudents.fulfilled, (state, action) => {  
  state.status = 'succeeded';  
  state.list.push(...action.payload); // spread the array  
})
```

อะไรคือ bug และควรเขียนอย่างไรแทน?

คำตอบ:

ปัญหา

push(...payload) ต่อท้ายทุกครั้งที่ทำให้เกิดข้อมูลซ้ำ การ fetch ครั้งที่สองจะทำให้ลิสต์เป็นสองเท่า

แก้ไข

```
state.list = action.payload; // replace - not push
```

Session 4 — สรุป

แนวคิดหลักที่ครอบคลุมใน Session 4:



Redux เป็น synchronous

Thunk middleware สกัดกัน function actions เพื่อให้ async code รันก่อน dispatch ผลลัพธ์



createAsyncThunk

ห่อ async payload creator และสร้าง action types แบบ pending / fulfilled / rejected อัตโนมัติ



extraReducers builder

จัดการ actions จากภายนอก slice — สาม lifecycle actions — ด้วย .addCase()



รูปแบบฟิลด์ status

'idle' → 'loading' → 'succeeded' | 'failed' ขับเคลื่อน loading/error UI ทั้งหมด



การรวม mockapi.io

AcadeMate อ่านและเขียนข้อมูลสดผ่าน hosted REST API โดยไม่ต้องมี backend เอง



unwrap() สำหรับข้อผิดพลาดใน component

ใช้ try/catch ใน components ขณะ dispatch thunks — สะอาดกว่าการตรวจสอบ action type

ตัวอย่างล่วงหน้า — Session 5: Selectors ขั้นสูงและประสิทธิภาพ

Session 5 ต่อยอดจากรากฐานนี้โดยปรับปรุงวิธีที่ components อ่านจาก Redux store

`createSelector`

Selectors แบบ memoized ที่คำนวณใหม่เฉพาะเมื่อ input เปลี่ยน — ป้องกัน re-render ที่ไม่จำเป็น

`createEntityAdapter`

โครงสร้าง state แบบ normalized สำหรับ collections: byId + allIds — ค้นหา $O(1)$ สำหรับ students, courses, grades

รูปแบบ `selectorsSlice`

รวมการคำนวณ derived ทั้งหมดไว้ใน `selectors.js` และทดสอบแยกต่างหาก

การวัดประสิทธิภาพ

ใช้ React DevTools Profiler + Redux DevTools ระบุและแก้ไขจุดที่ re-render บ่อย

เอกสารอ้างอิงและอ่านเพิ่มเติม — Session 4

แหล่งข้อมูล	URL	ใช้ในสไลด์
RTK: createAsyncThunk	https://redux-toolkit.js.org/api/createAsyncThunk	Slides 8–11, Lab Steps 1–4
RTK: extraReducers	https://redux-toolkit.js.org/api/createSlice#extrareducers	Slides 13–14, Lab Step 1
Redux: Middleware	https://redux.js.org/understanding/history-and-design/middleware	Slides 4–5
Redux Essentials Part 5	https://redux.js.org/tutorials/essentials/part-5-async-logic	Full Session 4
Redux Essentials Part 6	https://redux.js.org/tutorials/essentials/part-6-performance-normalization	Preview — Session 5
mockapi.io Docs	https://mockapi.io/docs	Lab Step 1 (API setup)
RTK Query Overview (preview)	https://redux-toolkit.js.org/rtk-query/overview	Preview — Session 6
React: useEffect	https://react.dev/reference/react/useEffect	Slide 19 (fetch on mount)
MDN: Fetch API	https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch	Slides 10–11 (thunks)